



HY340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

ΠΑΝΕΠΙΣΤΗΜΙΟ ΚΡΗΤΗΣ,
ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ,
ΤΜΗΜΑ ΕΠΙΣΤΗΜΗΣ ΥΠΟΛΟΓΙΣΤΩΝ

```
VAR i:Integer;  
FUNCTION(Symbol) replicate  
    x = (function(x,y){return x+y;});  
    class DelFunctor: public std::unary_function<
```

ΔΙΔΑΣΚΩΝ

Αντώνιος Σαββίδης



HY340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

Διάλεξη 5η ΣΥΝΤΑΚΤΙΚΗ ΑΝΑΛΥΣΗ - II

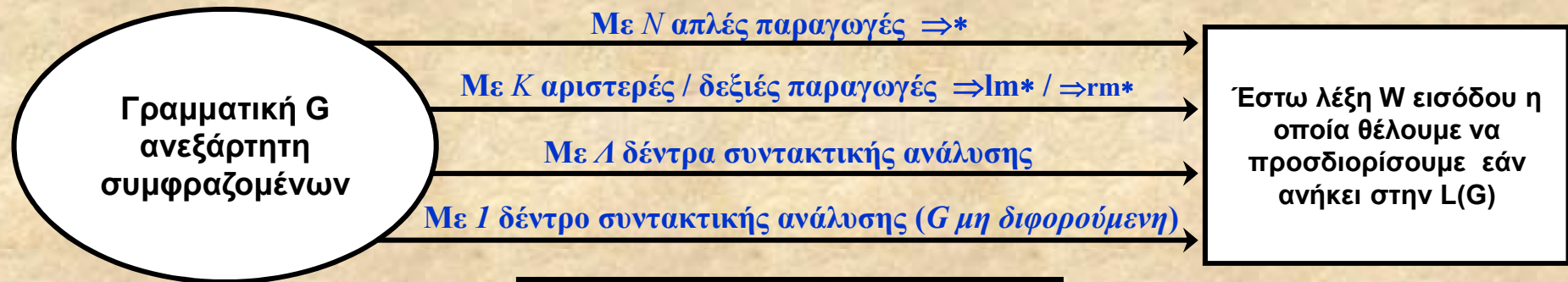


Περιεχόμενα

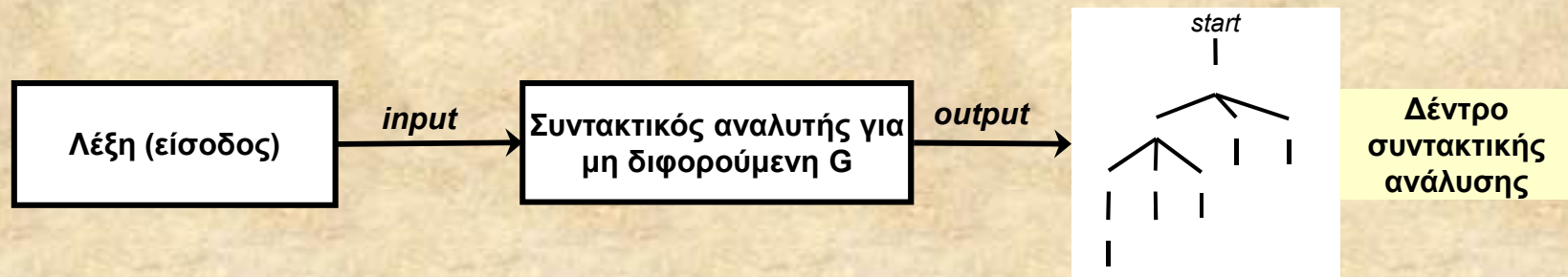
- *Κατασκευή συντακτικών αναλυτών*
- Καθοδική συντακτική ανάλυση
- Αναδρομική καθοδική συντακτική ανάλυση
- Εξάλειψη αριστερής γραμματικής αναδρομής
- Καθοδική ανάλυση με πρόβλεψη
- Συμπεράσματα



Κατασκευή συντακτικών αναλυτών (1/4)



$$W \in L(G) \Rightarrow N \geq K \geq \Lambda \geq 1$$



Πώς κατασκευάζουμε τον συντακτικό αναλυτή; αυτό το ερώτημα θα απαντήσουμε στη συνέχεια



Κατασκευή συντακτικών αναλυτών (2/4)

- Η κατασκευή συντακτικών αναλυτών έγκειται γενικά στον προσδιορισμό αλγορίθμων οι οποίοι
 - δεδομένης **λέξης** εισόδου μίας συγκεκριμένης μη διφορούμενης γραμματικής
 - μπορούν να δημιουργούν διαδοχικά, διαβάζοντας ένα-ένα τα γράμματα της λέξης
 - το μοναδικό **δέντρο συντακτικής ανάλυσης** το οποίο αντιστοιχεί στην **παραγωγή** της λέξης από τη γραμματική αυτή
- Εάν ένα τέτοιο δέντρο δεν υπάρχει για μία λέξη τότε αυτή δεν ανήκει στη γλώσσα που παράγει η αντίστοιχη γραμματική

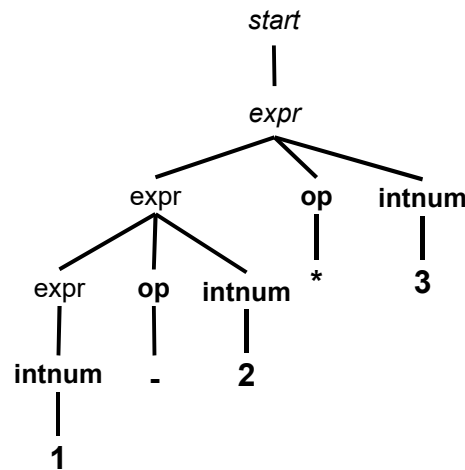


Κατασκευή συντακτικών αναλυτών (3/4)

Το δέντρο ανάλυσης για τη λέξη
1-2*3 της παρακάτω γραμματικής

$S \rightarrow expr$
 $expr \rightarrow expr \text{ op } intnum$
 $expr \rightarrow intnum$
 $expr \rightarrow (expr)$

$op = + \mid - \mid * \mid /$
 $intnum = digit^+$



- Ανάλογα με τη σειρά με την οποία κατασκευάζονται οι κόμβοι του **συντακτικού δέντρου** διακρίνουμε κυρίως δύο βασικές μεθόδους
 - Καθοδική συντακτική ανάλυση – *top down parsing*, όταν το δέντρο κατασκευάζεται **από τη ρίζα** $\rightarrow$ **προς τα φύλλα**
 - Ανοδική συντακτική ανάλυση – *bottom up parsing*, όταν το δέντρο κατασκευάζεται **από τα φύλλα** $\rightarrow$ **προς τη ρίζα**
- Οι καθοδικοί αναλυτές είναι πολύ διαδεδομένοι διότι μπορούν να πετύχουν μεγάλη ταχύτητα συντακτικής ανάλυσης
- ενώ οι ανοδικοί γιατί μπορούν να απευθύνονται σε μεγαλύτερη οικογένεια γραμματικών και γιατί υπάρχουν πολλά έτοιμα εργαλεία κατασκευής.

Κατασκευή συντακτικών αναλυτών (4/4)

- Η θεμελιώδης σχέση μεταξύ των γραμματικών, συντακτικών αναλυτών, λέξεων εισόδου και δέντρων συντακτικής ανάλυσης αποτυπώνεται στο παρακάτω σχήμα





Περιεχόμενα

- Κατασκευή συντακτικών αναλυτών
- *Καθοδική συντακτική ανάλυση*
- Αναδρομική καθοδική συντακτική ανάλυση
- Εξάλειψη αριστερής γραμματικής αναδρομής
- Καθοδική ανάλυση με πρόβλεψη
- Συμπεράσματα



Καθοδική συντακτική ανάλυση (1/10)

- Θα μελετήσουμε μία τεχνική καθοδικής κατασκευής συντακτικού δέντρου με εφαρμογή διαδοχικών γραμματικών παραγωγών
- Στην αμέσως επόμενη ενότητα θα δούμε πως η τεχνική αυτή μετουσιώνεται στην πράξη σε υλοποίηση ενός καθοδικού συντακτικού αναλυτή
 - *με όχι και τόσο εύκολη κατασκευή...*



Καθοδική συντακτική ανάλυση (2/10)

Η καθοδική κατασκευή ενός συντακτικού δέντρου εφαρμόζεται με αφετηρία τη ρίζα βάζοντας το όνομα του αρχικού μη τερματικού συμβόλου S , εκτελώντας επαναληπτικά τα εξής βήματα, με στόχο τον παράλληλο προσδιορισμό μίας παραγωγής και ταυτόχρονα την κατασκευή του συντακτικού δέντρου:

- Εάν το φύλλο a είναι μη τερματικό σύμβολο A , επέλεξε μία από τις παραγωγές του A , εφάρμοσέ τη και δημιούργησά τους αντίστοιχους κόμβους-παιδιά του a , από αριστερά προς δεξιά, για κάθε ένα από τα σύμβολα στο δεξιό τμήμα της επιλεγμένης παραγωγής.
- Συνέχισε έως ότου όλα τα φύλλα του δέντρου είναι τερματικά σύμβολα, οπότε θεώρησε το δέντρο ολοκληρωμένο.
- Εάν το δέντρο ολοκληρώθηκε, διάβασε από αριστερά προς τα δεξιά τη λέξη εισόδου (σύνολο από tokens), συγκρίνοντας με τα τερματικά σύμβολα που βρίσκονται στα φύλλα του δέντρου από αριστερά προς δεξιά. **dfs, μόνο τα φύλλα**
 - Εάν όλα κάνουν *match*, το συντακτικό δέντρο είναι δέντρο ανάλυσης της λέξης εισόδου.

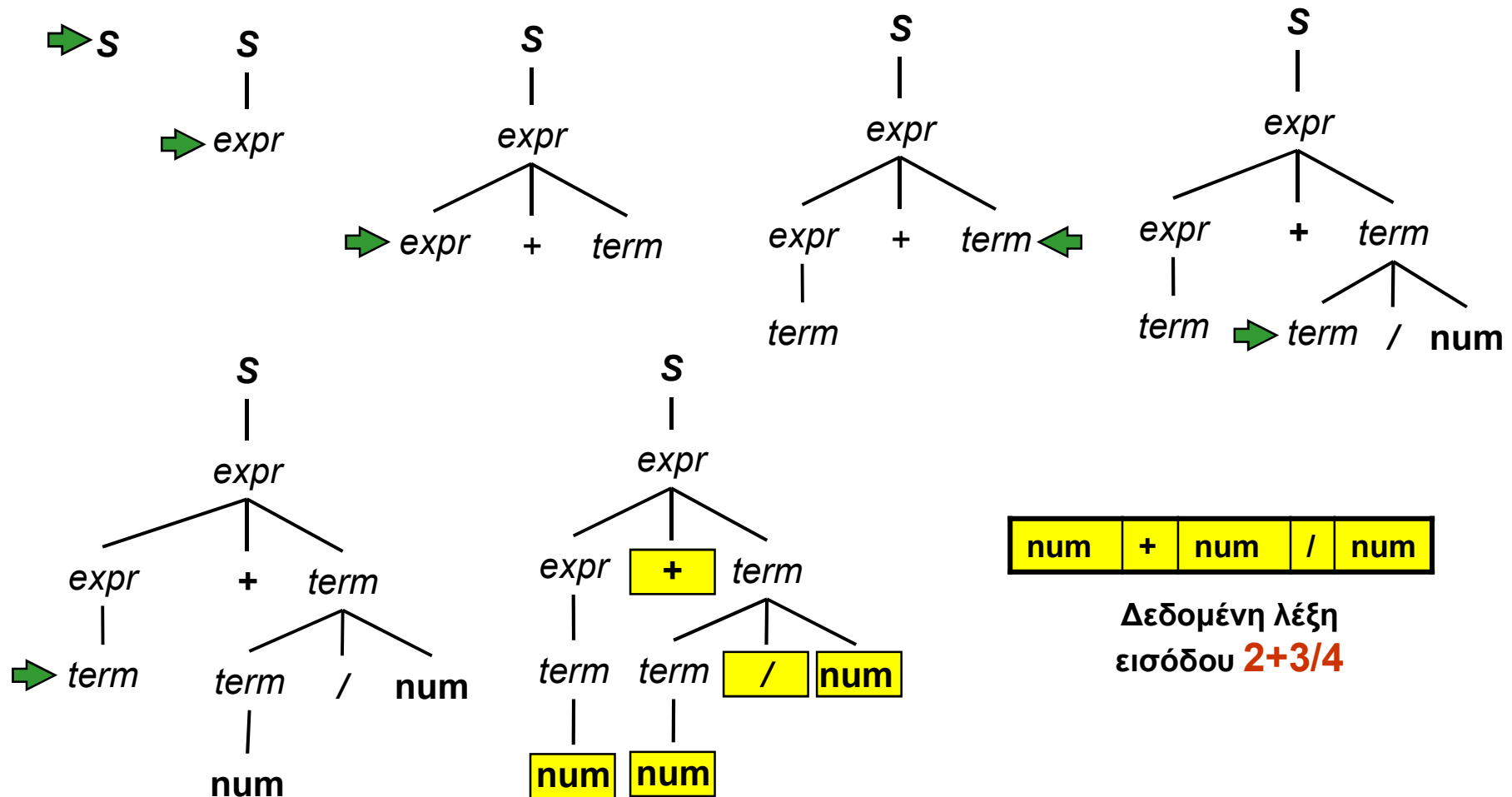
Η γραμματική που θα έχουμε σαν παράδειγμα

S	$\rightarrow expr$
$expr$	$\rightarrow expr + term$
$expr$	$\rightarrow expr - term$
$expr$	$\rightarrow term$
$term$	$\rightarrow term * num$
$term$	$\rightarrow term / num$
$term$	$\rightarrow num$

•Ο αλγόριθμος αυτός εξασφαλίζεται ότι βρίσκει πάντα το συντακτικό δέντρο εάν αυτό υπάρχει?
•Εάν αποτύχει με την επιλογή του δέντρου, με ποιο τρόπο συνεχίζει με την κατασκευή?
•Το κλειδί της επιτυχίας είναι η «σωστή» επιλογή των παραγωγών για την επέκταση του δέντρου.



Καθοδική συντακτική ανάλυση (3/10)





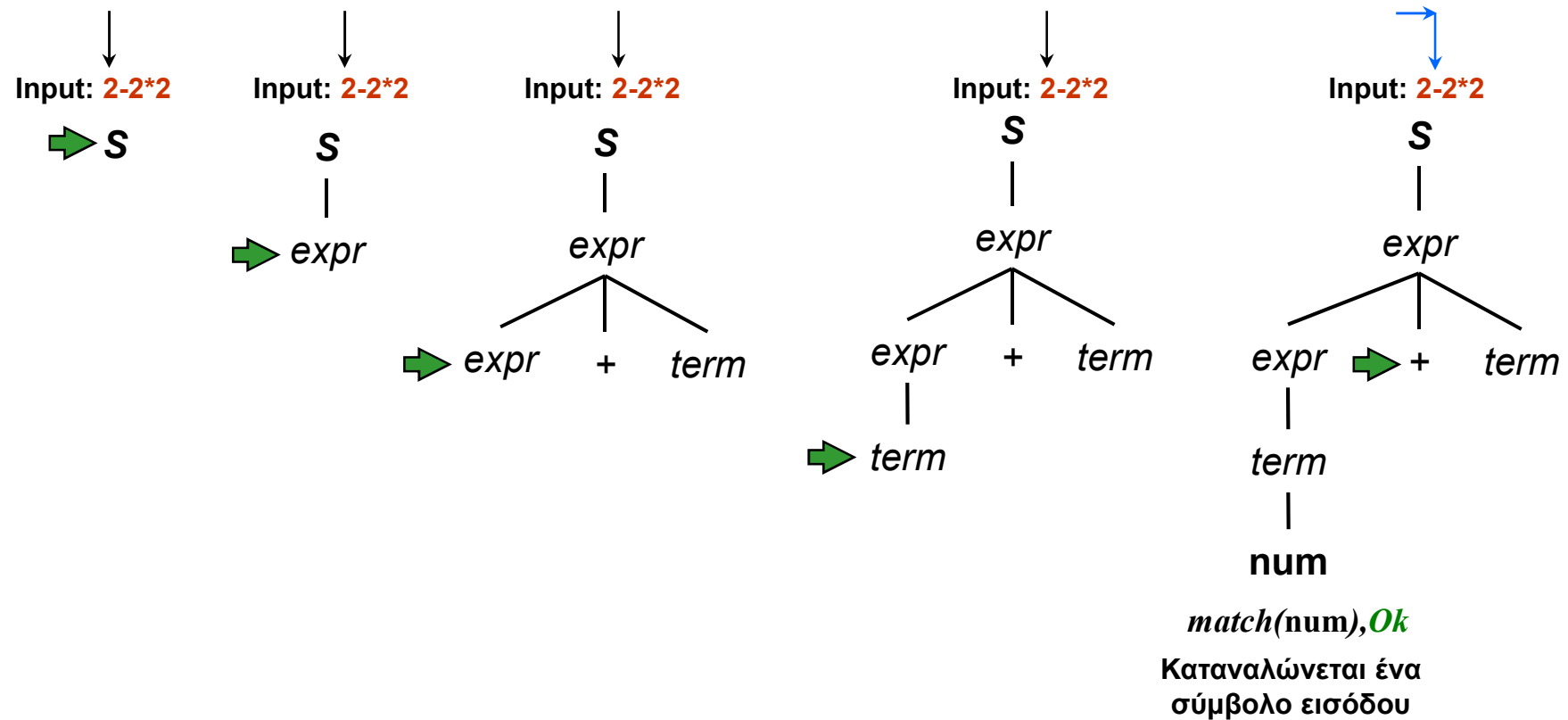
Καθοδική συντακτική ανάλυση (4/10)

Θα κάνουμε δύο σημαντικές εξειδικεύσεις στον αλγόριθμο, τόσο για να έχουμε συγκεκριμένη σειρά επεξεργασίας των κόμβων, όσο και για να εισάγουμε λογική σε περίπτωση αποτυχίας με μία παραγωγή:

- Η πρώτη είναι να εφαρμόζει αριστερές παραγωγές, που σημαίνει ότι δεν επεξεργαζόμαστε έναν κόμβο έως ότου για τον αριστερότερο έχει εφαρμοστεί ανάλυση μέχρι τερματικό σύμβολο και έχει γίνει *match* για το εκάστοτε σύμβολο εισόδου.
- Η δεύτερη εξειδίκευση είναι να μαρκάρουμε τις παραγωγές που επιλέγουμε για ένα μη τερματικό κόμβο:
 - Σε περίπτωση αποτυχίας (*failed match*) να δοκιμάζει μία νέα παραγωγή,
 - ενώ εάν δεν μπορεί να πετύχει για καμία παραγωγή στον εκάστοτε κόμβο, να επιστρέφει στον πατρικό κόμβο, θεωρώντας την εκάστοτε πατρική παραγωγή ως *failed*.

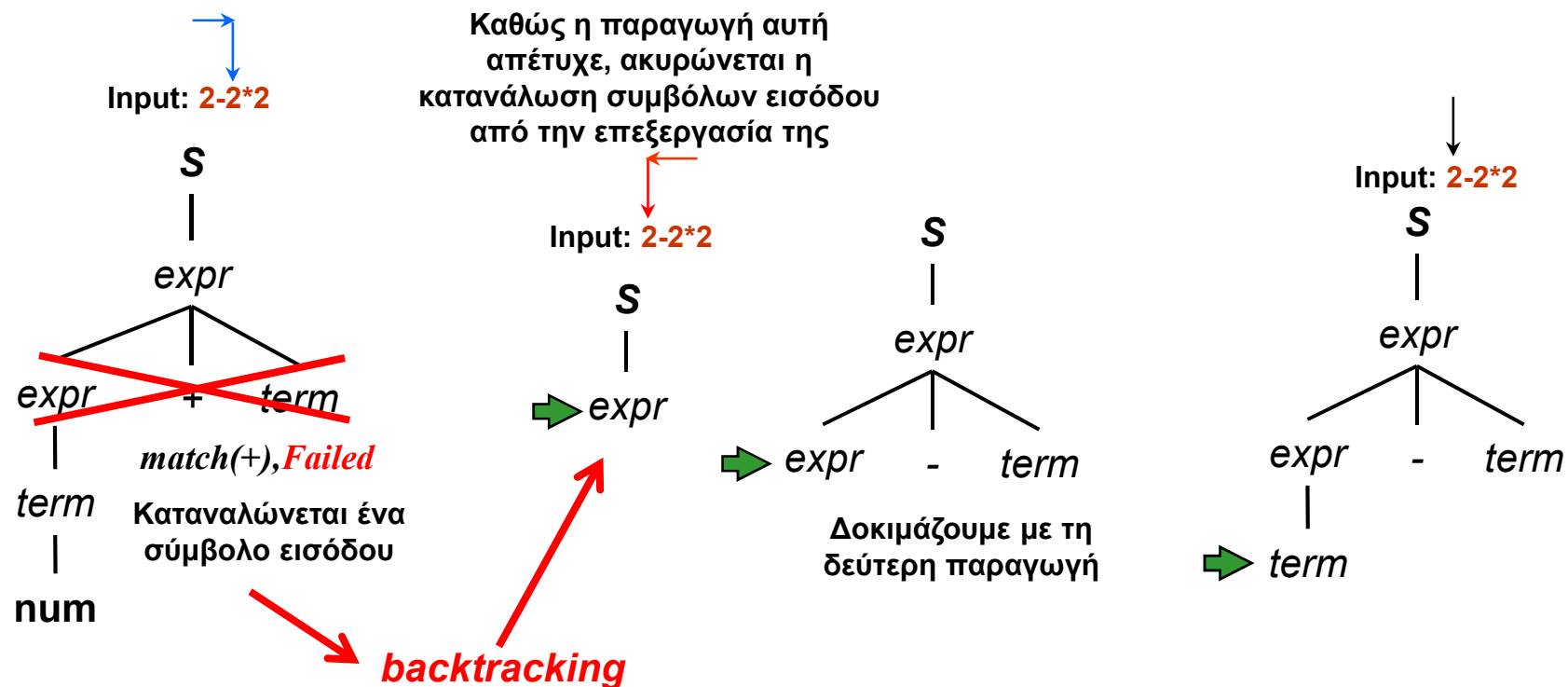


Καθοδική συντακτική ανάλυση (5/10)



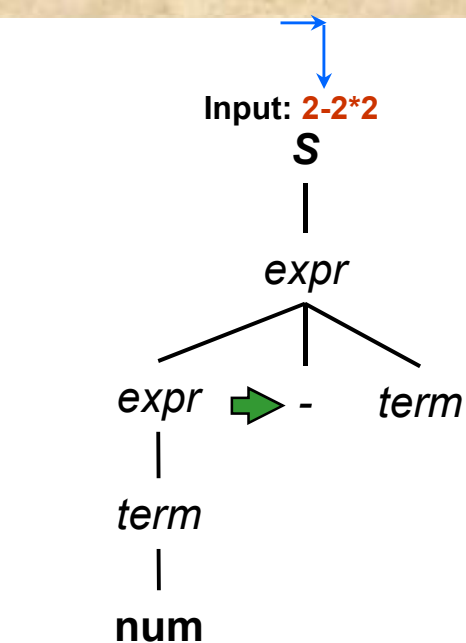


Καθοδική συντακτική ανάλυση (6/10)



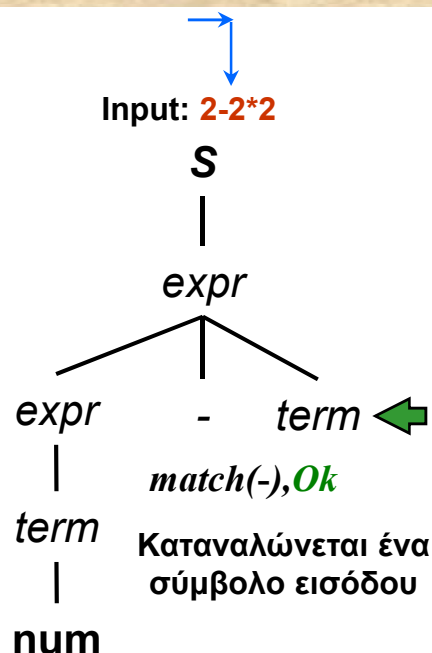


Καθοδική συντακτική ανάλυση (7/10)



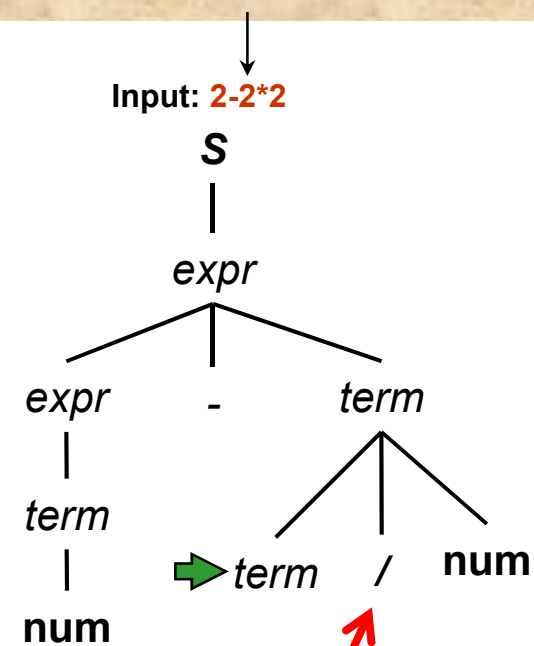
match(num), Ok

Καταναλώνεται ένα
σύμβολο εισόδου



match(-), Ok

Καταναλώνεται ένα
σύμβολο εισόδου



Υποπτευόμαστε ότι θα έχουμε
και άλλο backtracking λόγω του
ότι δεν θα κάνει match το /. Και
συνεχίζουμε με τον ίδιο τρόπο.



Καθοδική συντακτική ανάλυση (8/10)

```
topdown(A) {  
    u = mark_input_position();  
    if A είναι τερματικό σύμβολο then {  
        if match(A) then return true;  
    }  
    else {  
        found = false;  
        for  $\forall \Pi_i$  παραγωγή  $A \rightarrow \Pi_i$  και όσο found = false do {  
            Έστω  $\Pi_i$  παραγωγή της μορφής  $\alpha_1\alpha_2...\alpha_k$   
            found = true;  
            for  $\forall \alpha_i$  και όσο found=true do  
                found = topdown( $\alpha_i$ );  
            if found = false then  
                retract_input_till_position(u);  
        }  
        if found = true then return true;  
    }  
    retract_input_till_position(u);  
    return false;  
}
```

•Ο αλγόριθμος ξεκινά με *topdown(S)* και συνθήκη αναγνώρισης της λέξης «να επιστρέψει true και να μην υπάρχει άλλο σύμβολο στην είσοδο».

•Ο αλγόριθμος αυτός είναι η γενική μορφή του top down parsing.

Όπως φαίνεται, η επιλογή της εκάστοτε εναλλακτικής παραγωγής Π_i γίνεται απλώς με τη σειρά, χωρίς καμία ιδιαίτερη «εξυπνάδα» η «γνώση».

•Ο αλγόριθμος αυτός δεν είναι εξασφαλισμένο ότι τερματίζει, καθώς μπορούμε να βρούμε περιπτώσεις στις οποίες **ενώ υπάρχει parse tree και έχουμε πεπερασμένη είσοδο, η εκτέλεση δεν περατώνεται ποτέ!**



Καθοδική συντακτική ανάλυση (9/10)

- Ο προηγούμενος αλγόριθμος μας καθοδηγεί στα εξής βήματα καθοδικής ανάλυσης :
 - ξεκίνησε πρώτα με το αρχικό σύμβολο S και κάνε $topdown_parse(S)$
 - για κάθε εναλλακτική παραγωγή A του S δοκίμασε $topdown_parse(A)$
 - ◆ αν πετύχει κάποια τότε πέτυχε η συντακτική ανάλυση του S
 - με τον ίδιο τρόπο $topdown_parse(\alpha_i)$ για όλα τα σύμβολα $\alpha_1 \dots \alpha_n$ του κανόνα $A \rightarrow \alpha_1 \dots \alpha_n$ από αριστερά προς τα δεξιά
 - ◆ εάν για κάποιο α_i αποτύχει η ανάλυση θεώρησε ότι αποτύχαμε με αυτή την παραγωγή
 - η λογική αυτή εφαρμόζεται αναδρομικά ως ότου έχουμε $topdown_parse(\beta)$ με β τερματικό σύμβολο
 - ◆ οπότε γίνεται *matching* με το επόμενο σύμβολο εισόδου



Καθοδική συντακτική ανάλυση (10/10)

- Αυτός ο αναδρομικός αλγόριθμος μας βοηθά να σκιαγραφήσουμε την εξής μέθοδο κατασκευής καθοδικού αναλυτή:
 - Υλοποιούμε μία ξεχωριστή συνάρτηση *parse* για κάθε ξεχωριστό μη-τερματικό σύμβολο A της γραμματικής
 - **Κάθε τέτοια συνάρτηση θα καλεί τις αντίστοιχες συναρτήσεις για κάθε ένα από τα σύμβολα που εμφανίζονται στο δεξί τμήμα της παραγωγής του A**
 - Εάν έχουμε εναλλακτικές παραγωγές για το A αυτές θα δοκιμάζονται (καλούνται) με τη σειρά εμφάνισης τους στον ορισμό του A
 - Για τα τερματικά σύμβολα απλώς ελέγχουμε για matching με το εκάστοτε επόμενο σύμβολο εισόδου



Περιεχόμενα

- Κατασκευή συντακτικών αναλυτών
- Καθοδική συντακτική ανάλυση
- *Αναδρομική καθοδική συντακτική ανάλυση*
- Εξάλειψη αριστερής γραμματικής αναδρομής
- Καθοδική ανάλυση με πρόβλεψη
- Συμπεράσματα



Αναδρομική καθοδική ανάλυση (1/8)

- Θα παρουσιάσουμε πρώτα τον τυποποιημένο τρόπο κατασκευής των συναρτήσεων καθοδικής ανάλυσης για όλα τα σύμβολα.
 - Θα χρειαστούμε τους παρακάτω ορισμούς βοηθητικών συναρτήσεων και καθολικών μεταβλητών για να έχουμε *unlimited look-ahead*

```
array  tokens;  
integer size=0, curr=0;  
  
nexttoken() {  
    if curr = size then {  
        tokens[size] = gettoken();  
        size = size+1;  
    }  
    curr = curr+1;  
    return tokens[curr-1];  
}
```

Ποιο είναι το index του επόμενου token. Εάν είναι ίδιο με το size, χρειαζόμαστε lexical analysis, αλλιώς το έχουμε ήδη διαβάσει καθώς «κάποιος» πριν το έκανε retract.

```
if (curr == size)  
    tokens[size++] = gettoken();  
return tokens[curr++];
```

```
mark_input_position() { return curr; }  
retract_input_till_position (integer n) { curr = n; }
```



Αναδρομική καθοδική ανάλυση (2/8)

```
unput () { retract_input_till_position(size-1 ); }  
match (t) {  
    if nexttoken() = t return true;  
    else { unput(); return false; }  
}
```

Προσοχή, step-back to
πολύ μία θέση!
Δηλαδή unput()...unput()
ισοδύναμο με μία unput()

Η κατασκευή των συναρτήσεων γίνεται με επαγωγή ως προς το είδος των γραμματικών κανόνων:

- Για κάθε τερματικό σύμβολο a η αναγνώριση σημαίνει ότι η κλήση $match(a)$ επιστρέφει **true**.
- Για κάθε μη τερματικό σύμβολο A το οποίο έχει ξεχωριστές παραγωγές $A \rightarrow A_i$, $0 \leq i < n$ με A_i να συμβολίζει την ακολουθία συμβόλων της παραγωγής, υλοποιείται η συνάρτηση A ως εξής:

```
A() { Για τις εναλλακτικές παραγωγές  
    u = mark_input_position();  
    if A1() or A2() or...or An() = true then  
        return true;  
    retract_input_till_position(u);  
    return false;  
}
```

Αρκεί να πετύχουμε
αναγνώριση κάποιας
εκ των παραγωγών



Αναδρομική καθοδική ανάλυση (3/8)

Η κατασκευή της κάθε A_i συνάρτησης, με A_i γραμματικό κανόνα $\beta_1\beta_2...\beta_k$ ακολουθίας γραμματικών συμβόλων, ορίζεται ομοίως:

```
 $A_i()$  { Για μια συγκεκριμένη παραγωγή  
     $u = \text{mark\_input\_position}()$  ;  
    if  $\beta_1()$  and  $\beta_2()$  and... and  $\beta_n()$  = then  
        return true;  
    retract_input_till_position( $u$ );  
    return false;  
}
```

Πρέπει να πετύχουμε
αναγνώριση όλων
των γραμματικών
συμβόλων

- Αν β_i τερματικό σύμβολο, τότε $\beta_i()$ { **return** match(β_i); }, αλλιώς ακολουθείται αναδρομικά ο ίδιος κανόνας για την κατασκευή της β_i καθώς αντιστοιχεί σε μη τερματικό γραμματικό σύμβολο.

Θα χρησιμοποιήσουμε
αυτήν τη γραμματική
(τύποι της γλώσσας
Pascal) ως παράδειγμα

<i>type</i>	→ <i>simple</i> <i>pointer</i> <i>array</i>
<i>simple</i>	→ integer char <i>range</i>
<i>range</i>	→ num dotdot num
<i>array</i>	→ array [<i>range</i>] of <i>type</i>



Αναδρομική καθοδική ανάλυση (4/8)

```
int match (token_t t) {  
    if (t==nexttoken())  
        return 1;  
    else  
        { unput(); return 0; }  
}
```

Η match η οποία σε περίπτωση αποτυχίας κάνει αυτομάτως και *unput*.

```
#define INTEGER_TOKEN 1  
#define CHAR_TOKEN 2  
#define ARRAY_TOKEN 3  
#define NUM_TOKEN 4  
#define DOTDOT_TOKEN 5  
#define OPENBR_TOKEN 6  
#define CLOSEBR_TOKEN 7  
#define OF_TOKEN 8
```

Οι σταθερές με τις οποίες αναγνωρίζουμε το κάθε ξεχωριστό τύπο token

```
int type_f (void);  
int simple_f (void);  
int range_f (void);  
int pointer_f (void);  
int array_f (void);
```

Οι συναρτήσεις καθοδικής ανάλυσης για τα μη-τερματικά σύμβολα

```
#define MARK() int u = mark_input_position();  
#define FAIL() retract_input_till_position(u); return 0;  
  
int type_f (void) {  
    MARK();  
    if (simple_f() || pointer_f() || array_f())  
        return 1;  
    FAIL();  
}
```

Δοκιμάζονται οι εναλλακτικές παραγωγές με τη σειρά (μόνο εάν η προηγούμενη δεν πέτυχε)



Αναδρομική καθοδική ανάλυση (5/8)

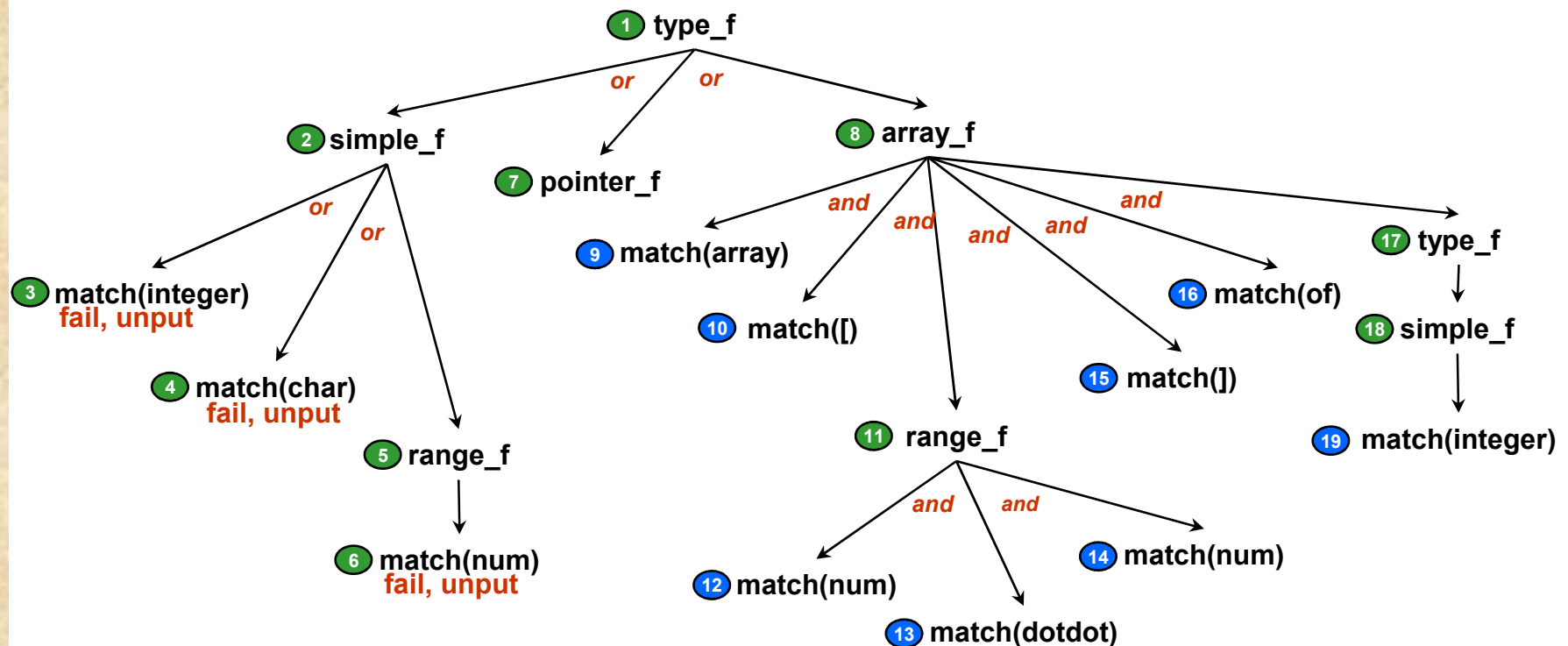
```
int simple_f (void) {  
    MARK();  
    if (match(INTEGER_TOKEN) || match(CHAR_TOKEN) || range_f())  
        return 1;  
    FAIL();  
}  
  
int range_f (void) {  
    MARK();  
    if (match(NUM_TOKEN)    &&  
        match(DOTDOT_TOKEN) &&  
        match(NUM_TOKEN))  
        return 1;  
    FAIL();  
}  
  
int array_f (void) {  
    MARK();  
    if (match(ARRAY_TOKEN)    &&  
        match(OPENBR_TOKEN)  &&  
        range_f()             &&  
        match(CLOSEBR_TOKEN) &&  
        match(OF_TOKEN)      &&  
        type_f())  
        return 1;  
    FAIL();  
}
```

Για έναν γραμματικό κανόνα $\beta_1\beta_2\ldots\beta_k$ πρέπει να αναγνωριστούν όλα τα σύμβολα με τη σειρά παράθεσης.



Αναδρομική καθοδική ανάλυση (6/8)

Οι αριθμημένες ετικέτες δείχνουν τη σειρά με την οποία γίνονται οι κλήσεις των συναρτήσεων



Λέξη εισόδου

array [1 .. 10] of integer

Οι κλήσεις στις οποίες έχουμε επιτυχές matching συμβόλου εισόδου με τις οποίες προχωράει η είσοδος στο επόμενο σύμβολο



Αναδρομική καθοδική ανάλυση (7/8)

- Η κατασκευή ενός αναδρομικού καθοδικού αναλυτή – recursive descent parser (RDP), φαίνεται ότι είναι σχετικά αυτοματοποιημένη και εύκολη διαδικασία για οποιαδήποτε γραμματική
- Ωστόσο υπάρχουν πολλές γραμματικές για τις οποίες ο RDP που προκύπτει δεν δουλεύει στην πράξη, καθώς εισέρχεται σε ατέρμονη αναδρομή
- Οι γραμματικές αυτές είναι οι ονομαζόμενες **αριστερά αναδρομικές – left recursive**
 - όσες έχουν κάποιο μη τερματικό σύμβολο **A** για το οποίο υπάρχει παραγωγή **A =>+ Aα** για κάποια γραμματική ακολουθία **α**
 - ◆ δηλ. με μία ή περισσότερες παραγωγές ένα γραμματικό σύμβολο παράγει λέξη που έχει το ίδιο το σύμβολο στα αριστερά της λέξης
 - π.χ. η γραμματική για αριθμητικές εκφράσεις μπορεί να οδηγήσει σε μία παραγωγή της μορφής **expr =>+ expr+10**, άρα θεωρείται αριστερά αναδρομική



Αναδρομική καθοδική ανάλυση (8/8)

The left recursion problem

```
#define PLUS_TOKEN      9
#define MINUS_TOKEN    10

int expr_f1 (void); /* "expr + term" */
int expr_f2 (void); /* "expr - term" */
int expr_f3 (void); /* "term" */
int term_f  (void);
int term_f1 (void); /* "term * num" */
int term_f2 (void); /* "term / num" */
int term_f3 (void); /* "num" */

int expr_f (void) {
    MARK();
    if (expr_f1() ||
        expr_f2() ||
        expr_f3())
        return 1;
    FAIL();
}

int expr_f1 (void) {
    MARK();
    if (expr_f() &&
        match(PLUS_TOKEN) &&
        term_f())
        return 1;
    FAIL();
}
```

$S \rightarrow \text{expr}$
 $\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term}$
 $\text{term} \rightarrow \text{term} * \text{num} \mid \text{term} / \text{num} \mid \text{num}$

•Ο κώδικας του RDP που αντιστοιχεί σε αυτή η γραμματική φαίνεται αριστερά και κάτω. Είναι προφανές ότι λόγω της αριστερής γραμματικής αναδρομής, η *expr_f* τελικά καλεί τον εαυτό της πάντα, γεγονός που οδηγεί εξασφαλισμένα σε ατέρμονη αναδρομή.

•Τι κάνουμε σε αυτή την περίπτωση? εξάλειψη αριστερής αναδρομής από τη γραμματική.

```
int expr_f2 (void) {
    MARK();
    if (expr_f() &&
        match(MINUS_TOKEN) &&
        term_f())
        return 1;
    FAIL();
}

int expr_f3 (void) {
    MARK();
    if (term_f())
        return 1;
    FAIL();
}
```




Ένθετο

```
epxr_f () {  
    MARK();  
    if (term() || expr_plus() || expr_minus())  
        return 1;  
    FAIL();  
}  
  
term() {  
    MARK();  
    if (  
        term_parenthesis() ||  
        match(NUM_TOKEN) ||  
        term_div() ||  
        term_mul()  
    )  
        return 1;  
    FAIL();  
}  
  
expr_plus() {  
    MARK();  
    if (expr() && match(PLUS_TOKEN) && term())  
        return 1;  
    FAIL();  
}
```

Μπορούμε αν αποφύγουμε το left recursions με διαφορετική σειρά κλήσεων, γνωρίζοντας τη δομή της γραμματικής, ώστε να επιβάλλουμε πρώτα ανάγνωση της εισόδου



Περιεχόμενα

- Κατασκευή συντακτικών αναλυτών
- Καθοδική συντακτική ανάλυση
- Αναδρομική καθοδική συντακτική ανάλυση
- *Εξάλειψη αριστερής γραμματικής αναδρομής*
- Καθοδική ανάλυση με πρόβλεψη
- Συμπεράσματα

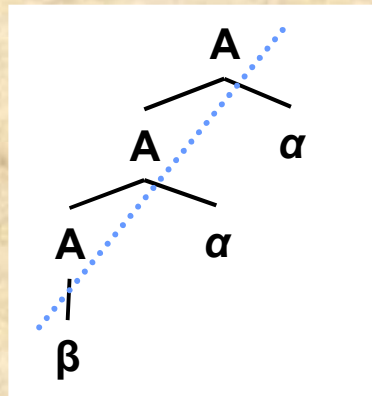
Εξάλειψη αριστερής αναδρομής (1/7)

Εξάλειψη αριστερής αναδρομής

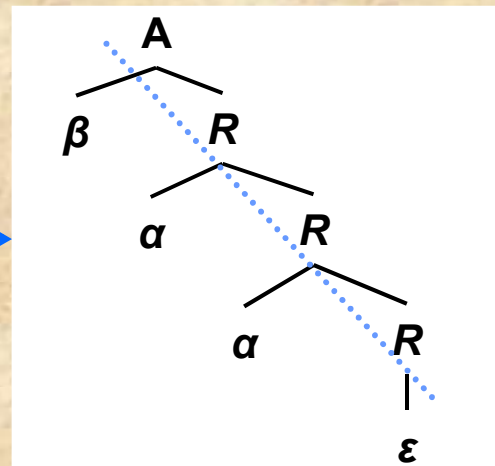
Έστω αριστερά αναδρομικές παραγωγές της μορφής $A \rightarrow A\alpha \mid \beta$, για α και β ακολουθίες συμβόλων. Τότε οι κανόνες αυτοί αντικαθίστανται από τους παρακάτω τρεις κανόνες, στους οποίους η αριστερή αναδρομή δίνει τη θέση της σε δεξιά αναδρομή:

- $A \rightarrow \beta R$
- $R \rightarrow \alpha R$
- $R \rightarrow \varepsilon$

Παρατηρούμε ότι οι φράσεις που παράγει το A αρχίζουν πάντοτε με β



Το παλιό parse tree για την παραγωγή της ακολουθίας βαα



Το νέο parse tree για την παραγωγή της ακολουθίας βαα

•Φαίνεται και η επίδραση της αριστερής ή δεξιάς αναδρομής στη δομή του δέντρου συντακτικής ανάλυσης για την παραγωγή ακολουθιών γραμματικών συμβόλων

•**Αριστερή κατιούσα** κατασκευή και **δεξιά κατιούσα** κατασκευή αντίστοιχα

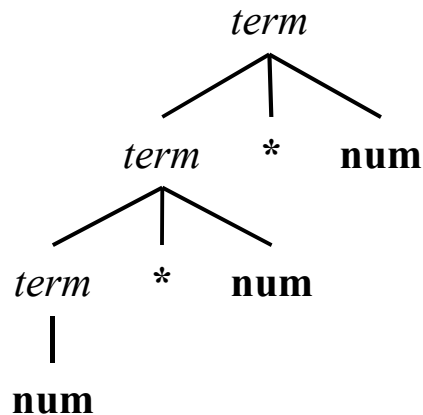


Εξάλειψη αριστερής αναδρομής (2/7)

$term \rightarrow term * num$
 $term \rightarrow term / num$
 $term \rightarrow num$
 α είναι $*num$ και $/ num$ και β είναι num

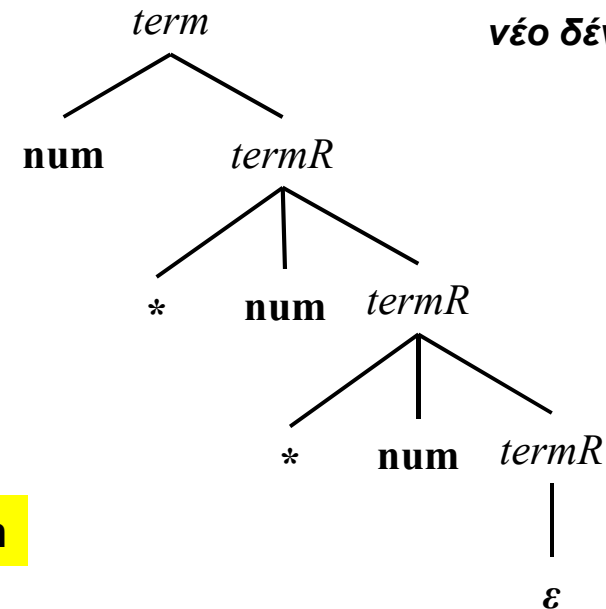
$term \rightarrow num termR$
 $termR \rightarrow * num termR$
 $termR \rightarrow / num termR$
 $termR \rightarrow \varepsilon$

αρχικό δέντρο



num*num*num

νέο δέντρο



Η γραμματική γίνεται περισσότερο δυσνόητη, ενώ το παραγόμενο δέντρο συντακτικής ανάλυσης γίνεται λιγότερο πρόσφορο για επεξεργασία (χρειάζεται περισσότερο hacking στις αναδρομικές συναρτήσεις ανάλυσης για να παράγουμε τα δέντρα που χρειαζόμαστε).



Εξάλειψη αριστερής αναδρομής (3/7)

- Η προηγούμενη τεχνική είναι ικανή να εξαλείψει την άμεση αριστερή αναδρομή από μία γραμματική αλλά όχι και την έμμεση, η οποία ωστόσο είναι το ίδιο καταστροφική για την κατασκευή αναδρομικού καθοδικού αναλυτή όσο και η άμεση. Π.χ. έστω παρακάτω γραμματική:

$$\begin{aligned} S &\rightarrow A \alpha \mid b \\ A &\rightarrow A c \mid S d \mid \varepsilon \end{aligned}$$

- Ενώ το S δεν έχει άμεσο αριστερά αναδρομικά κανόνα, εντούτοις είναι αριστερά αναδρομικό, καθώς: $S \Rightarrow A\alpha \Rightarrow \underline{Sda}$. Ένα πιο αναγνωρίσιμο παράδειγμα έμμεσης αριστερής αναδρομής είναι το εξής:

$$\begin{aligned} \text{expr} &\rightarrow \text{func} (\text{args}) \\ \text{func} &\rightarrow \text{expr} \end{aligned}$$

$$\boxed{\text{expr}} \Rightarrow \text{func} (\text{args}) \Rightarrow \boxed{\text{expr}} (\text{args})$$



Εξάλειψη αριστερής αναδρομής (4/7)

Εκτός ύλης

Είσοδος: Γραμματική G χωρίς κύκλους και κενές παραγωγές. Η εξάλειψη κενών παραγωγών και κύκλων από μία γραμματική θα δούμε πως μπορεί να εφαρμοστεί αργότερα.

Έξοδος: Ισοδύναμη γραμματική χωρίς αριστερή αναδρομή.

Βάλτε σε κάποια διάταξη $A_1, A_2, \dots, A_n$ όλα τα μη τερματικά σύμβολα.

for $i=1$ to n do {

for $j=1$ to $i-1$ do {

Αντικατέστησε κάθε $A_i \rightarrow A_j \gamma$ από τις παραγωγές
 $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$, όπου $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$
είναι όλες οι παραγωγές του A_j

}

Εξάλειψε όλες τις άμεσες αριστερές αναδρομές από τις παραγωγές του A_i

}



Αλγόριθμος εξάλειψης - ένθετο (1/5)

■ Outer loop (on counter i)

- $A_1 A_2 A_3 \dots A_N$

Εκτός ύλης

■ Inner loop (on counter j)

- none when at A_1
- A_1 when at A_2 (inline A_1 in A_2)
- $A_1 A_2$ when at A_3 (inline A_1, A_2 in A_3)
- $A_1 A_2 A_3$ when at A_4 (inline A_1, A_2, A_3 in A_4)
- $A_1 A_2 A_3 \dots A_{i-1}$ when at A_i (inline $A_1, A_2, \dots, A_{i-1}$ in A_i)
- $A_1 A_2 A_3 \dots A_{N-1}$ when at A_N (inline $A_1, A_2, \dots, A_{N-1}$ in A_N)



Αλγόριθμος εξάλειψης - ένθετο (2/5)

ΕΠΕΞΗΓΗΣΗ

Έστω $A \rightarrow B\alpha$ και $B \rightarrow \gamma$. Ορίζουμε ως αναδίπλωση του B στην παραγωγή του A την αντικατάσταση του B απευθείας από τις παραγωγές του. Δηλ. η αναδίπλωση του B στο A οδηγεί στην αντικατάσταση του $A \rightarrow B\alpha$ από τον $A \rightarrow \gamma\alpha$. Στην προκειμένη περίπτωση το B είναι επιπλέον αριστερής θέσης.

Ο αλγόριθμος εξάλειψης αριστερής αναδρομής βάζει σε μία τυχαία διάταξη τα σύμβολα, π.χ. $A_1, A_2, \dots, A_n$, και έπειτα κάνει τα εξής:

- Εξάλειψη άμεσης αριστερής αναδρομής στο A_1
- Αναδιπλώνει στις παραγωγές του A_2 ($i=2$) το A_1 ($j:1 \rightarrow 2-1$) αριστερή θέσης
 - Έπειτα εξάλειψη άμεσης αριστερής αναδρομής
- Αναδιπλώνει στις παραγωγές του A_3 ($i=3$) με τη σειρά τα A_2, A_1 ($j:1 \rightarrow 3-1$) αριστερής θέσης
 - Έπειτα εξάλειψη άμεσης αριστερής αναδρομής
- ...
- Αναδιπλώνει στις παραγωγές του A_n ($i=n$) με τη σειρά τα $A_{n-1}, \dots, A_2, A_1$ ($j:1 \rightarrow n-1$) αριστερής θέσης
 - Έπειτα εξάλειψη άμεσης αριστερής αναδρομής

Εκτός ύλης



Αλγόριθμος εξάλειψης - ένθετο (3/5)

ΠΑΡΑΔΕΙΓΜΑ (1/2)

Εκτός ύλης

$$\begin{array}{lll} A \rightarrow BC & B \rightarrow CA \mid b & C \rightarrow AB \mid a \\ A_1 & A_2 & A_3 \end{array}$$

Η γραμματική είναι προβληματική καθώς έχω παραγωγή που οδηγεί σε έμμεση αριστερή αναδρομή:

$$\boxed{A} \Rightarrow BC \Rightarrow CAC \Rightarrow \boxed{A}BAC$$

$$i=1$$

Eliminate left recursion from $A \rightarrow BC$ (καμία ενέργεια)

$$i=2$$

$j=1$ Unfold A in B (καμία ενέργεια, παραμένει το B)

Eliminate left recursion from $B \rightarrow CA \mid b$ (καμία ενέργεια)



Αλγόριθμος εξάλειψης - ένθετο (4/5)

ΠΑΡΑΔΕΙΓΜΑ (2/2)

$i=3$

Εκτός ύλης

$j=1$ Unfold A in C

$$C \rightarrow AB \mid a \Rightarrow C \rightarrow BCB \mid a$$

$j=2$ Unfold B in C

$$C \rightarrow BCB \mid a \Rightarrow C \rightarrow CACB \mid bCB \mid a$$

Eliminate left recursion from C , αφού μετατρέψουμε για ευκολία το $C \rightarrow CACB \mid bCB \mid a$ στη μορφή στα δεξιά:

$$C_1 \rightarrow ACB \qquad C \rightarrow C_2 R \qquad C \rightarrow bCB R \mid aR$$

$$C_2 \rightarrow bCB \mid a \Rightarrow R \rightarrow C_1 R \mid \varepsilon \Rightarrow R \rightarrow ACBR \mid \varepsilon$$

$$C \rightarrow CC_1 \mid C_2$$

Εισαγάγουμε τα βοηθητικά μη τερματικά σύμβολα C_1 , C_2 και R

Η τελική μετασχηματισμένη γραμματική είναι πλέον:

$$A \rightarrow BC$$

$$B \rightarrow CA \mid b$$

$$C \rightarrow bCB R \mid aR$$

$$R \rightarrow ACBR \mid \varepsilon$$



Αλγόριθμος εξάλειψης - ένθετο (5/5)

ΣΧΕΣΗ ΜΕ ΕΠΙΛΥΣΗ ΣΥΣΤΗΜΑΤΟΣ ΕΞΙΣΩΣΕΩΝ

Εκτός ύλης

Αυτός ο τρόπος είναι ανάλογος με τη λύση συστήματος εξισώσεων. Π.χ., στο παρακάτω σύστημα έστω η διάταξη $x (A_1)$, $y (A_2)$, $z (A_3)$. Πρέπει να εφαρμόσω αναδιπλώσεις: x στο y , x στο z και y στο z .

$x = y + 3$	Αναδίπλωση του x στο y . 0	Αναδίπλωση του y στο z . 1
$y = -z + 1$	Αναδίπλωση του x στο z . 1	$z = -(-z + 1 + 3) + 2z + 6$
$z = -x + 2z + 6$	$z = -(y + 3) + 2z + 6$ <i>1^ο βήμα</i>	$z = -2$, επίλυση (εξάλειψη) <i>2^ο βήμα</i>

$$A \rightarrow BC \quad B \rightarrow CA \mid b \quad C \rightarrow AB \mid a$$

Έστω η διάταξη A, B, C . Άρα θα κάνω τις εξής αναδιπλώσεις αριστερής θέσης και εξαλείψεις άμεσης αριστερής αναδρομής:

A στο B : καμία

εξάλειψη: καμία

A στο C : $C \rightarrow \underline{BC}B \mid a$

εξάλειψη: καμία

B στο C : $C \rightarrow \underline{C}ACB \mid \underline{b}CB \mid a$

εξάλειψη: όπως πριν



Εξάλειψη αριστερής αναδρομής (5/7)

Εξάλειψη κενών παραγωγών. Έστω παραγωγή της μορφής $A \rightarrow \varepsilon$. Τότε υπάρχουν δύο περιπτώσεις (τα α και γ είναι ακολουθία γραμματικών συμβόλων):

- Αυτός είναι ο μοναδικός κανόνας για το A , άρα $\exists B$ μη τερματικό σύμβολο με $B \rightarrow \alpha A \gamma$. Για κάθε τέτοια περίπτωση αλλάζουμε τον κανόνα σε $B \rightarrow \alpha \gamma$.
- Αλλιώς, κάθε κανόνας $A \rightarrow \alpha A \gamma$ αντικαθίσταται από τους: $A \rightarrow \alpha \gamma$ και $A \rightarrow \alpha A \gamma$. Ο $A \rightarrow \varepsilon$ πλέον αφαιρείται.

Εξάλειψη κύκλων. Έστω G με κύκλους της μορφής $A \Rightarrow^+ A$. Τότε θα πρέπει η τελευταία παραγωγή του κύκλου να είναι $\alpha \beta \Rightarrow A$ όπου $\alpha \neq \varepsilon \vee \beta \neq \varepsilon$, αλλιώς τα α ή β θα παράγουν κάποια τερματικά σύμβολα, άρα δεν μπορεί να είναι τότε $A \Rightarrow^+ A$. Αρκεί να εξαλείψουμε λοιπόν τις όποιες αυτές κενές παραγωγές.

Εκτός ύλης



Εξάλειψη αριστερής αναδρομής (6/7)

- Εάν σκεφτούμε λίγο το πώς δουλεύει **ο βασικός κατασκευαστικός** αλγόριθμος αναδρομικής καθοδικής ανάλυσης
- Παρατηρούμε ότι **βασίζεται σε απεριόριστο *look-ahead***
- Αλλά και σε back-tracking, όταν κάποια γραμματική παραγωγή που δοκιμάζεται δεν παράγει τα σύμβολα εισόδου
- Πρακτικά, το απεριόριστο look-ahead στην είσοδο δεν είναι πρόβλημα, καθώς μπορεί να υλοποιηθεί αρκετά εύκολα



Εξάλειψη αριστερής αναδρομής (7/7)

- Ωστόσο στο back-tracking δεν έχουμε απλώς επιστροφές αναδρομικών κλήσεων που ενορχηστρώνονται εύκολα και input retraction που είναι τετριμμένο
- Ουσιαστικά απαιτείται και ακύρωση των όποιων ενεργειών μετάφρασης έχουν ήδη εκτελεστεί και οι οποίες φυσιολογικά ενσωματώνονται μέσα στις αναδρομικές συναρτήσεις συντακτικής ανάλυσης
- Δεν είναι καθόλου προφανές το πόσο εύκολο, δύσκολο ή ακόμη και εφικτό είναι να υλοποιηθούν τέτοιες είδους ακυρωτικές πράξεις (*undoing translation actions*)
- Για το λόγο αυτό γεννήθηκε η μέθοδος κατασκευής καθοδικών αναλυτών με πρόβλεψη την οποία και θα εφαρμόσουμε



Περιεχόμενα

- Κατασκευή συντακτικών αναλυτών
- Καθοδική συντακτική ανάλυση
- Αναδρομική καθοδική συντακτική ανάλυση
- Εξάλειψη αριστερής γραμματικής αναδρομής
- *Καθοδική ανάλυση με πρόβλεψη*
- Συμπεράσματα



Καθοδική ανάλυση με πρόβλεψη (1/14)

- Στο βασικό αλγόριθμο καθοδικής ανάλυσης το πρόβλημα ήταν η αδυναμία επιλογής του σωστού εναλλακτικού κανόνα για επεξεργασία και επέκταση του δέντρου
 - αυτό είναι ουσιαστικά που μας οδηγεί σε backtracking
- Μπορούμε να εφαρμόσουμε καθοδική ανάλυση με πρόβλεψη όταν:
 - για το εκάστοτε μη τερματικό σύμβολο στον αλγόριθμο καθοδικής ανάλυσης
 - παρατηρώντας μόνο το επόμενο token εισόδου, δηλ. με look-ahead 1
 - μπορούμε επακριβώς να επιλέξουμε τη σωστή από τις εναλλακτικές γραμματικές παραγωγές



Καθοδική ανάλυση με πρόβλεψη (2/14)

- Αυτό στην ειδική περίπτωση της αναδρομικής καθοδικής ανάλυσης σημαίνει
 - για κάθε συνάρτηση μη τερματικού συμβόλου
 - η κλήση των συναρτήσεων ανάλυσης για τις εναλλακτικές παραγωγές του συμβόλου αυτού
 - γίνεται επιλεκτικά με έλεγχο του επόμενου συμβόλου εισόδου
- Προφανώς θα πρέπει η γραμματική να μου επιτρέπει μόνο με look-ahead 1 να μπορώ προσδιορίζω τη σωστή παραγωγή
- ➔ *Ας δούμε ένα τέτοιο κατασκευαστικό παράδειγμα μετατρέποντας τον αρχικό αναδρομικό αναλυτή της γραμματικής δηλώσεων τύπων Pascal*



Καθοδική ανάλυση με πρόβλεψη (3/14)

```
char* tokenClasses[]={
    "type prefix"
    "integer",
    "char",
    "array",
    "number",
    "..",
    "[",
    "]",
    "of"
    /* etc */
};

void error (token_t t)
{ printf("token %s expected.\n", tokenClasses[t]); }

int lookAhead = 0;
void match (token_t t) {
    if (!lookAhead)
        lookAhead = nexttoken();
    if (lookAhead==t)
        lookAhead = nexttoken();
    else
        error(t);
}
```

Μικρή αλλαγή της match

```
void Parse (void) {
    lookAhead = nexttoken();
    type_f();
}
```

```
void simple_f (void) {
    if (lookAhead==INTEGER_TOKEN)
        match(INTEGER_TOKEN);
    else
        if (lookAhead==CHAR_TOKEN)
            match(CHAR_TOKEN);
        else
            if (lookAhead==NUM_TOKEN) {
                match(NUM_TOKEN);
                match(DOTDOT_TOKEN);
                match(NUM_TOKEN);
            }
            else
                assert(0);
}
```

```
void type_f (void) {
    if (lookAhead==INTEGER_TOKEN ||
        lookAhead==CHAR_TOKEN ||
        lookAhead==NUM_TOKEN)
        simple_f();
    else
        if (lookAhead==POINTER_TOKEN)
            pointer_f();
        else
            if (lookAhead==ARRAY_TOKEN)
                array_f();
            else
                error(0);
}
```

```
void range_f (void) {
    if (lookAhead==NUM_TOKEN) {
        match(NUM_TOKEN);
        match(DOTDOT_TOKEN);
        match(NUM_TOKEN);
    }
    else
        assert(0);
}

void array_f (void) {
    if (lookAhead==ARRAY_TOKEN) {
        match(ARRAY_TOKEN);
        match(OPENBR_TOKEN);
        range_f();
        match(CLOSEBR_TOKEN);
        match(OF_TOKEN);
        type_f();
    }
    else
        assert(0);
}
```

Παρατηρήστε ότι οι ουσιαστικές αλλαγές στον RDP είναι: (α) η εισαγωγή ελέγχου του look-ahead για την επιλογή του εναλλακτικού κανόνα, (β) η πιστοποίηση του look-ahead στην περίπτωση ακολουθίας γραμματικών συμβόλων.



Καθοδική ανάλυση με πρόβλεψη (4/14)

- Η ανάλυση με πρόβλεψη βασίζεται σε πληροφορία σχετικά με το ποια είναι τα πρώτα σύμβολα που μπορούν να παραχθούν από το δεξί τμήμα μίας παραγωγής
- Έτσι, στον προηγούμενο αναλυτή, η ανάλυση του *type* επιλέγει την παραγωγή *simple* εάν το look-ahead είναι ένα από τα **integer**, **char** ή **num**, γνωρίζοντας ότι μόνο το *simple* παράγει πρώτα αυτά τα σύμβολα. Αυτή η πληροφορία μπορεί να τυποποιηθεί ως εξής:

Έστω μία παραγωγή $A \rightarrow \alpha$, με α να είναι ακολουθία γραμματικών συμβόλων. Ορίζουμε ως $\text{FIRST}(\alpha)$ το σύνολο των τερματικών συμβόλων τα οποία εμφανίζονται πρώτα στις λέξεις που παράγει το α . Εάν $\alpha \Rightarrow^* \varepsilon$ τότε $\varepsilon \in \text{FIRST}(\alpha)$. Π.χ.

$$\text{FIRST}(\text{simple}) = \{ \text{integer}, \text{char}, \text{num} \}$$

$$\text{FIRST}(\text{array}) = \{ \text{array} \}$$

$$\text{FIRST}(\text{type}) = \text{FIRST}(\text{simple}) \cup \text{FIRST}(\text{array}) \cup \text{FIRST}(\text{pointer})$$



Καθοδική ανάλυση με πρόβλεψη (5/14)

- Στην πράξη, πολλά δεξιά τμήματα παραγωγών αρχίζουν με τερματικά σύμβολα, γεγονός που καθιστά τον προσδιορισμό του FIRST συνόλου σε σχετικά εύκολο.
- Τα FIRST σύνολα χρειάζονται σε παραγωγές της μορφής $A \rightarrow \alpha \mid \beta$. Για την κατασκευή ενός RDP χωρίς back-tracking απαιτείται $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$. Τότε η επιλογή της κατάλληλης παραγωγής ορίζεται μοναδικά μέσω του look-ahead συμβόλου ως εξής:

if look-ahead $\in \text{FIRST}(\alpha)$ διάλεξε το α **else**

if look-ahead $\in \text{FIRST}(\beta)$ διάλεξε το β **else**

error

- Οι εναλλακτικές παραγωγές με ε στο δεξί τμήμα θέλουν ειδική αντιμετώπιση. Οι παραγωγές αυτές ελέγχονται πάντα τελευταίες, εάν για όλες τις υπόλοιπες το look-ahead δεν ανήκει στα αντίστοιχα FIRST sets. Π.χ.

funcall $\rightarrow$ *func* (*actualargs*);

actualargs $\rightarrow$ *exprlist* $\mid \varepsilon$

actualargs() {

if *lookahead* $\in \text{FIRST}(\text{exprlist})$ **then**

exprlist();

/* Καμία ενέργεια, δηλ. ε -παραγωγή */

}



Καθοδική ανάλυση με πρόβλεψη (6/14)

Το πρόβλημα των κοινών
προθεμάτων στις εναλλακτικές παραγωγές

Υπάρχουν περιπτώσεις στις οποίες για ένα μη-τερματικό σύμβολο έχουμε εναλλακτικές γραμματικές παραγωγές με κάποιο κοινό πρόθεμα, ώστε μόνο με το look-ahead να μην είναι εφικτή η επιλογή της σωστής παραγωγής. Π.χ., έστω οι παραγωγές:

$$\begin{array}{lcl} stmt & \rightarrow & \text{if } expr \text{ then } stmt \text{ else } stmt \\ & | & \text{if } expr \text{ then } stmt \end{array}$$

Τότε τη στιγμή που διαβάζουμε το σύμβολο **if** ισχύει:

$$\begin{array}{l} \text{if} \in \text{FIRST}(\text{if } expr \text{ then } stmt \text{ else } stmt) \text{ και} \\ \text{if} \in \text{FIRST}(\text{if } expr \text{ then } stmt) \end{array}$$

Όμως τότε δεν μπορούμε να εφαρμόσουμε predictive parsing, καθώς τα FIRST σύνολα των εναλλακτικών παραγωγών δεν είναι όλα ξένα μεταξύ τους. Η λύση είναι να μετασχηματίσουμε τη γραμματική σε ισοδύναμη στην οποία όμως έχουμε εξαλείψει όλες τις περιπτώσεις κοινού προθέματος. Η τακτική αυτή λέγεται αριστερή παραγοντοποίηση – left factoring, και γίνεται με τον εξής αλγόριθμο.



Καθοδική ανάλυση με πρόβλεψη (7/14)

Αλγόριθμος
left factoring

- Για κάθε μη τερματικό σύμβολο A βρες το μεγαλύτερο πρόθεμα α το οποίο είναι κοινό σε δύο ή περισσότερες εναλλακτικές παραγωγές του A . Εάν $\alpha \neq \varepsilon$, δηλ. πρόκειται για μη τετριμμένο κοινό πρόθεμα, αντικατέστησε όλες τις παραγωγές της μορφής $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$, με γ να αναπαριστά όλες τις υπόλοιπες εναλλακτικές παραγωγές που δεν έχουν πρόθεμα α , με τις εξής παραγωγές:

$$A \rightarrow \alpha R \mid \gamma \quad \text{και} \quad R \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n.$$

Παράδειγμα
εφαρμογής

- Π.χ., παρακάτω φαίνεται πως μετασχηματίζονται η αρχική γραμματική για if else εντολές σε μία που δεν έχει κοινά προθέματα. Βέβαια να μην ξεχνάμε ότι και οι δύο αυτές γραμματικές είναι διφορούμενες.

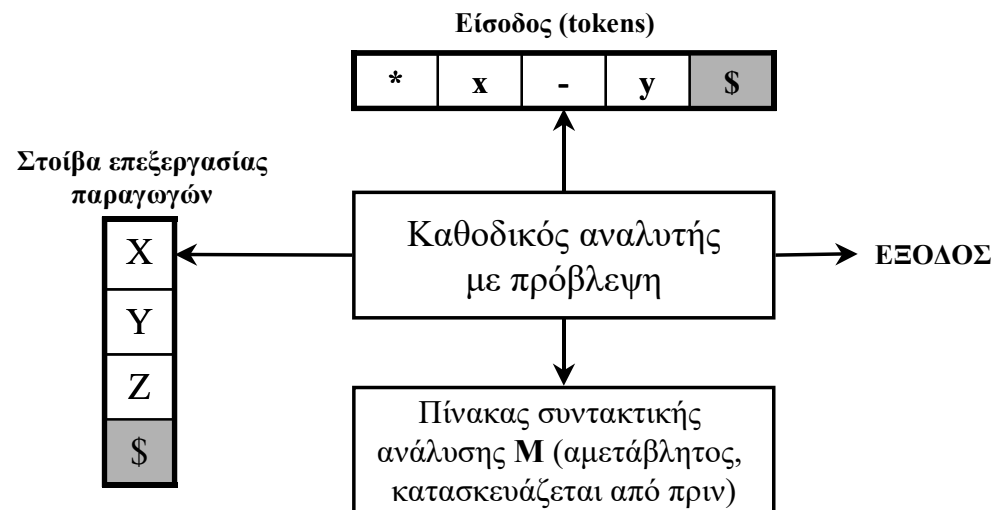
$$\begin{array}{ll} S & \rightarrow \text{if } expr \text{ then } stmt \mid \text{if } expr \text{ then } stmt \text{ else } stmt \mid a \\ expr & \rightarrow b \end{array}$$

$$\begin{array}{ll} S & \rightarrow \text{if } expr \text{ then } stmt \mid R \mid a \\ R & \rightarrow \text{else } stmt \mid \varepsilon \\ expr & \rightarrow b \end{array}$$

Καθοδική ανάλυση με πρόβλεψη (8/14)

Θα δούμε μία τεχνική κατασκευής ενός *predictive top-down parser* ο οποίος αντί να εμπεριέχει αναδρομικές κλήσεις συναρτήσεων για τα μη τερματικά σύμβολα, διατηρεί μία στοίβα με τις παραγωγές που επεξεργάζεται κατά την κατασκευή του δέντρου.

Καθοδική ανάλυση με πρόβλεψη χωρίς αναδρομή



Ο αναλυτής που θα κατασκευάσουμε έχει τη δομή του παραπάνω σχήματος και είναι οδηγούμενος από έναν πίνακα M τον οποίο τον έχουμε δημιουργήσει από πριν, για τη συγκεκριμένη γραμματική. Ο αναλυτής διαβάζει την είσοδο από ένα buffer, ο οποίος τερματίζει με το ειδικό σύμβολο `$` (end of input). Η στοίβα περιέχει μία ακολουθία από γραμματικά σύμβολα, με `$` να συμβολίζει τον πάτο της στοίβας (bottom of stack). Ο πίνακας συντακτικής ανάλυσης M είναι δισδιάστατος της μορφής $M[A, \alpha]$, όπου A είναι μη τερματικό σύμβολο της γραμματικής και α είναι τερματικό σύμβολο ή το `$`.

prediction table:
ενώ είμαστε στην $A()$ και με επόμενο token το α ποιά συνάρτηση θα κληθεί;



Καθοδική ανάλυση με πρόβλεψη (9/14)

Ο αναλυτής αυτός λειτουργεί βάση ενός τυποποιημένου αλγορίθμου ο οποίος σκιαγραφείται παρακάτω. Έστω ότι την εκάστοτε χρονική το σύμβολο εισόδου είναι το a και το γραμματικό σύμβολο X είναι στην κορυφή της στοίβας. Τότε:

1. Εάν X είναι τερματικό σύμβολο:
 - Εάν είναι $X = a$ και $a = \$$, ο αναλυτής τερματίζει και αναφέρει επιτυχή περάτωση της διαδικασίας συντακτικής ανάλυσης.
 - Εάν $X = a$ και $a \neq \$$, τότε ο αναλυτής κάνει pop το X από τη στοίβα και προχωράει στην ανάγνωση του επόμενου συμβόλου εισόδου.
 - Εάν $X \neq a$, τότε **error**.
2. Αλλιώς το X είναι μη τερματικό σύμβολο. Τότε ελέγχεται το $M[X, a]$.
 - Εάν έχει την τιμή **error**, τότε **error**.
 - Αλλιώς περιέχει μία γραμματική παραγωγή του X , έστω $X \rightarrow UVW$, δηλ. $M[X, a] = \{X \rightarrow UVW\}$. Αυτό ο αναλυτής το αντιμετωπίζει κάνοντας pop το X και push με τη σειρά W , V και U (δηλ. από δεξιά προς τα αριστερά). Σαν έξοδο μπορεί να βγάλει μήνυμα της παραγωγής που επεξεργάστηκε, αλλά και να εκτελέσει κάποια συνάρτηση (κώδικα) που μπορεί και αυτή να αποθηκεύεται στα στοιχεία πίνακα.



Καθοδική ανάλυση με πρόβλεψη (10/14)

Είσοδος: Μία λέξη w από N τερματικά σύμβολα και ένας πίνακας M για τη γραμματική G .

Έξοδος: Εάν $w \in L(G)$, την αριστερή παραγωγή του w , αλλιώς *error*.

Αλγόριθμος καθοδικής ανάλυσης
με πρόβλεψη καθοδηγούμενος από πίνακα.

```
token input[N+1];
integer i=0;      /* Θέση παρόντος συμβόλου εισόδου */
Βάλε όλα τα tokens της λέξης  $w$  με τη σειρά στο input.
input[N] = $;     /* Στο τέλος το $ */
push($, S);       /* Αρχικό σύμβολο της  $G$  στην κορυφή */

repeat forever {
     $X = top()$ ;  $\alpha = input[i]$ ;
    if isterminal( $X$ ) then
        if  $X = \alpha$  then
            if  $\alpha = \$$  then accept();
            else { pop();  $i=i+1$ ; }
        else error();
    else
        if  $M[X, \alpha] = X \rightarrow Y_1 Y_2 \dots Y_k$  then {
            pop();
            push( $Y_k, Y_{k-1}, \dots, Y_1$ ); /* δηλ.  $Y_1$  στην κορυφή */
            output(" $X \rightarrow Y_1 Y_2 \dots Y_k$ ");
        }
    else error();
}
```



Καθοδική ανάλυση με πρόβλεψη (11/14)

Σημαντικές επεξηγήσεις για το πώς λειτουργεί ο αλγόριθμος.

Σε ποια τεχνική που ήδη έχουμε περιγράψει βασίζεται αυτός ο αλγόριθμος? Αρκεί να παρατηρήσουμε τα εξής: όταν στη στοίβα είναι κάποιο μη τερματικό σύμβολο X και στην είσοδο το token a , ο αλγόριθμος συμβουλευεται το $M[X,a]$ για να αποφασίσει με ποια παραγωγή του X θα ασχοληθεί.

- Αυτό όμως θυμίζει ακριβώς την αναδρομική τεχνική στην οποία είχαμε τη συνάρτηση ανάλυσης για το X , όπου η κλήση της κατάλληλης συνάρτησης εναλλακτικής παραγωγής του X γινόταν βάση του ελέγχου «σε ποιο $\text{FIRST}(\beta_i)$ ανήκει το look-ahead», για $X \rightarrow \beta_1 | \beta_2 | \dots | \beta_n$.
- Η αλλιώς, δεδομένου τερματικού συμβόλου a και της παραγωγής X , θα κληθεί η συνάρτηση της β_i εάν $a \in \text{FIRST}(\beta_i)$. Αυτό σημαίνει ότι $M[X,a] = \beta_i : a \in \text{FIRST}(\beta_i)$.

Τώρα διακρίνεται ότι η στοίβα και ο πίνακας ανάλυσης εξομοιώνουν επακριβώς τις αναδρομικές κλήσεις των συναρτήσεων. Με αυτή την παρατήρηση, η κατασκευή του πίνακα δεν είναι πια δύσκολη, καθώς αποτυπώνει απλώς τη λογική των κλήσεων των επιμέρους συναρτήσεων αναδρομικού καθοδικού αναλυτή με πρόβλεψη – predictive recursive descend parser.



Καθοδική ανάλυση με πρόβλεψη (12/14)

type → *simple* | *pointer* | *array*
simple → **integer** | **char** | *range*
array → **array** [*range*] **of** *type*
pointer → ^ *type*

Parsing table και κινήσεις
 του καθοδικού αναλυτή
 για δεδομένη λέξη
 εισόδου, στη γραμματική
 των Pascal τύπων. Δείτε
 την αντιστοιχία με τον
 κώδικα του αναδρομικού
 αναλυτή με πρόβλεψη
 που είχαμε κατασκευάσει.

Μη τερματικά	Τερματικά σύμβολα εισόδου (tokens)							
	integer	char	array	num	..	[	]	of
<i>type</i>	→ <i>simple</i>	→ <i>simple</i>	→ <i>array</i>	→ <i>simple</i>				→ <i>pointer</i>
<i>simple</i>	→ integer	→ char		→ <i>range</i>				
<i>pointer</i>								→^ <i>type</i>
<i>array</i>			array [<i>range</i>] of <i>type</i>					
<i>range</i>				→ num .. num				

Στοιβά	Είσοδος	Έξοδος
\$ <i>type</i>	array [10..20] of ^ integer \$	<i>type</i> → <i>array</i>
\$ <i>array</i>	array [10..20] of ^ integer \$	<i>array</i> → array [<i>range</i>] of <i>type</i>
\$ <i>type</i> of] <i>range</i> [array	array [10..20] of ^ integer \$	
\$ <i>type</i> of] <i>range</i> [	[10..20] of ^ integer \$	
\$ <i>type</i> of] <i>range</i>	10..20] of ^ integer \$	<i>range</i> → num .. num
\$ <i>type</i> of] num .. num	10..20] of ^ integer \$	
\$ <i>type</i> of] num ..	..20] of ^ integer \$	
\$ <i>type</i> of] num	20] of ^ integer \$	
\$ <i>type</i> of]	] of ^ integer \$	
\$ <i>type</i> of	of ^ integer \$	
\$ <i>type</i>	^ integer \$	<i>type</i> → <i>pointer</i>
\$ <i>pointer</i>	^ integer \$	<i>pointer</i> →^ <i>type</i>
\$ <i>type</i> ^	^ integer \$	
\$ <i>type</i>	integer \$	<i>type</i> → <i>simple</i>
\$ <i>simple</i>	integer \$	<i>simple</i> → integer
\$ integer	integer \$	
\$	\$	<i>Accept</i>



Καθοδική ανάλυση με πρόβλεψη (13/14)

Η προηγούμενη κατασκευή δεν είναι ακόμη τυποποιημένη, αλλά βασιστήκαμε στη χρήση των FIRST sets για τον αναδρομικό καθοδικό αναλυτή. Επίσης, ακόμη δεν έχουμε τυποποιήσει την κατασκευή του FIRST set. Για τον ορισμό ενός τυποποιημένου αλγορίθμου κατασκευής του parsing table, χρειαζόμαστε _____

- Αλγόριθμο κατασκευής των συνόλων FIRST.
- Αλγόριθμο κατασκευής των συνόλων FOLLOW.
- Αλγόριθμο κατασκευής του predictive parsing table (βασίζεται στα δύο προηγούμενα).

Με τα παραπάνω ολοκληρώνεται η θεωρία κατασκευής των καθοδικών αναλυτών. Όμως η μέθοδος αυτή δεν είναι εφαρμόσιμη για όλων των ειδών τις γραμματικές. *Ειδικότερα, ένα έχουμε διαφορούμενες ή αριστερά αναδρομικές γραμματικές, ο πίνακας που προκύπτει έχει τουλάχιστον ένα στοιχείο για το οποίο ορίζονται εναλλακτικές παραγωγές.*



Καθοδική ανάλυση με πρόβλεψη (14/14)

Μία γραμματική για την οποία ο αλγόριθμος κατασκευής πίνακα καθοδικής ανάλυσης δεν προκαλεί πολλαπλούς ορισμούς των στοιχείων του πίνακα ονομάζεται LL(1).

- **L**, γιατί γίνεται parsed διαβάζοντας την είσοδο Left to right.
- **L**, γιατί είναι κατάλληλη για τον προσδιορισμό αριστερών παραγωγών – Leftmost derivations.
- **(1)**, γιατί μπορούμε να πετύχουμε τα παραπάνω έχοντας γνώση απλώς για ένα επόμενο σύμβολο εισόδου (ένα look-ahead).

- Δεν είναι εφικτή η μετατροπή όλων των γραμματικών σε LL(1) μορφή.
- Αρκετές φορές ακόμη και εάν η μετατροπή είναι εφικτή, η παραγόμενη γραμματική δεν είναι πρόσφορη για μετάφραση (τα δέντρα ανάλυσης που παράγονται δεν έχουν τη δομή που θα βόλευε).
- Επίσης μετατροπές ή επεκτάσεις στη γραμματική ενδέχεται να προκαλέσουν αρκετές μεταβολές στον πίνακα ή στην οργάνωση των αναδρομικών κλήσεων.



Καθοδική ανάλυση με πρόβλεψη και αριστερή αναδρομή (1/2)

$S \rightarrow \text{expr}$
 $\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term}$
 $\text{term} \rightarrow \text{term} * \text{prim} \mid \text{term} / \text{prim} \mid \text{prim}$
 $\text{prim} \rightarrow (\text{expr}) \mid \text{num}$

Αυτή είναι η γραμματική των αριθμητικών εκφράσεων με αριστερή προσεταιριστικότητα (δηλ. αριστερά αναδρομική) και προτεραιότητα τελεστών

```
int    lookAhead;  
struct T {};  
void    error    (char*);  
void    match    (int);  
T*      tree     (T* left, int binaryOp, T* right);  
T*      tree     (int unaryOp, T* child);  
T*      tree     (int token);
```

Μπορούμε να κατασκευάσουμε απευθείας έναν predictive RDP χωρίς να πειράξουμε τη γραμματική και με τρόπο που ο RDP συνεχίζει να είναι ευανάγνωστος και κατασκευάζει συντακτικά δέντρα που έχουν την επιθυμητή δομή;

```
T* EXPR (void);  
T* TERM (void);  
T* TERM_PRIM (void);  
T* PRIM_NUM (void);  
T* PRIM_PAR (void);
```

```
T* EXPR (void) {  
    T* left = TERM();  
    while (lookAhead == PLUS_TOKEN || lookAhead == MINUS_TOKEN) {  
        match(lookAhead);  
        left = tree(left, lookAhead, TERM()); // force left associative  
    }  
    return left;  
}
```




Καθοδική ανάλυση με πρόβλεψη και αριστερή αναδρομή (2/2)

```
T* TERM (void) {  
    T* left = TERM_PRIM();  
    while (lookAhead == MUL_TOKEN || lookAhead == DIV_TOKEN) {  
        match(lookAhead);  
        left = tree(left, lookAhead, TERM_PRIM()); // force left associative  
    }  
    return left;  
}
```

```
T* TERM_PRIM (void) {  
    if (lookAhead == OPENPAR_TOKEN)  
        return PRIM_PAR();  
    else  
        if (lookAhead == NUM_TOKEN)  
            return PRIM_NUM();  
        else  
            { error("'(' or number expected in 'prim'"); return nullptr; }  
}
```

```
T* PRIM_NUM (void)  
{ match(NUM_TOKEN); return tree(NUM_TOKEN); }  
  
T* PRIM_PAR (void) {  
    match(OPENPAR_TOKEN);  
    T* e = EXPR();  
    match(CLOSEPAR_TOKEN);  
    return e;  
}
```



Ένθετο (1/3)

Εκτός ύλης

```
FIRST(X) {           το X είναι ένα μόνο σύμβολο  
  if X is token c then return {c};  
  if X is  $\epsilon$  then return { $\epsilon$ };  
  X is  $Y_1|Y_2|\dots|Y_n$   
  return  $FIRST(Y_1) \cup \dots \cup FIRST(Y_n)$   
}  
  
FIRST(X_1X_2\dots X_n) {   το X είναι ακολουθία συμβόλων  
  result = {}, i=0  
  repeat {  
    i=i+1;  
    result = result  $\cup$  {FIRST(Xi) - { $\epsilon$ }};  
  } until i=n or not Xi=>*  $\epsilon$   
  if not  $X_1X_2\dots X_n \Rightarrow^* \epsilon$  then  
    result = result  $\cup$  { $\epsilon$ };  
  return result;  
}
```



Ένθετο (2/3)

Εκτός ύλης

```
FOLLOW(X) {  
    result = {}  
    if X = S then  
        result = result  $\cup$  {$};  
    foreach Y  $\rightarrow \alpha X \beta$  do {  
        if  $\beta$  begins with terminal c then  
            result = result  $\cup$  {c};  
        else {  
            if  $\beta$  begins with non-terminal then  
                result = result  $\cup$  FIRST( $\beta$ ) - { $\epsilon$ };  
            if  $\beta \rightarrow \epsilon$  or  $\beta \Rightarrow^* \epsilon$  then  
                result = result  $\cup$  FOLLOW(Y)  
        }  
    }  
    return result;  
}
```




Ένθετο (3/3)

Εκτός ύλης

```
LL(G) {  
  let  $M$  parsing table  
  foreach  $A \rightarrow \alpha \in G$  do {  
    foreach  $b \in FIRST(\alpha)$  do  
       $M[A, b] = A \rightarrow \alpha$   
    if  $\varepsilon \in FIRST(\alpha)$  then  
      foreach  $c \in FOLLOW(A)$ ,  $c$  token or  $\$$  do  
         $M[A, c] = A \rightarrow \alpha$   
  }  
  if an entry was written more than once then  
    throw " $G$  is not LL"  
  set undefined  $M$  entries to error  
  return  $M$   
}
```



Περιεχόμενα

- Κατασκευή συντακτικών αναλυτών
- Καθοδική συντακτική ανάλυση
- Αναδρομική καθοδική συντακτική ανάλυση
- Εξάλειψη αριστερής γραμματικής αναδρομής
- Καθοδική ανάλυση με πρόβλεψη
- *Συμπεράσματα*



Συμπεράσματα (1/5)

- Η καθοδική ανάλυση εμπλέκει τεχνικές που μπορούν να εφαρμοστούν με κάπως ελεγχόμενη δυσκολία.
- Για σχετικά απλές γραμματικές, με γνώση των προβλημάτων εξαρχής (*αριστερή αναδρομή, κοινά προθέματα, διαφορούμενοι κανόνες*) μπορεί να κατασκευαστεί ένας αναλυτής πλήρως από τον προγραμματιστή
- Γενικά μπορεί κάποιος να κάνει hack είτε τη γραμματική ή ακόμη και τα δέντρα ανάλυσης ώστε να εξυπηρετούν τους σκοπούς της μετάφρασης
- Εν ολίγοις, μία δυνατή κατασκευαστική τεχνική (ειδικά ο RDP που είναι πιο «εύπεπτος») που καλό είναι την γνωρίζετε – θα σας φανεί χρήσιμη



Συμπεράσματα (2/5)

```
void      Start (unsigned long timeout)
            { parseError = false; endTime = ugettim

bool      HasParseError (void)
            { return parseError; }

bool      Parse_KWD (util_ui16 kwd);
bool      Parse_STRING (char** value);
bool      Parse_INT (util_i64* value);
bool      Parse_BOOL (bool* value);
bool      Parse_UUID (util_ui8** uuid);

// Parsing server replies.
//
void      Parse_Terminator (void);
bool      Parse_ServLAddr (util_i64* id, util_i64* po
bool      Parse_ServAttrNameVal( char** name, char**
bool      Parse_ServAttrNameVals (AttrValueList* attr

bool      Parse_ServType (char** type)
```

```
bool Parser::Parse_INT (util_i64* value) {

bool      if (parseError)          // Propagate parse error.
bool      return false;

    if (!hasLookAhead) {
        if (!channel->ReceiveWithTimeout(&currToken, endTime)) {
            SetParseError("timeout while expecting 'INT'!");
            return false;
        }
    }
    else
        hasLookAhead = false;

    if (currToken.type != TokenTypeINT) {
        hasLookAhead = true;    // Will not consume this token.
        return false;
    }
    else {
        *value = currToken.token.intVal;
        return true;
    }
}
```

```
bool Parser::Parse_ServDesc (ServiceList* serviceList)

    if (parseError)          // Propagate parse error.
        return false;

    util_i64      id      = 0;
    util_i64      port    = 0;
    char*         type    = (char*) 0;
    AttrValueList* attrs  = new AttrValueList;

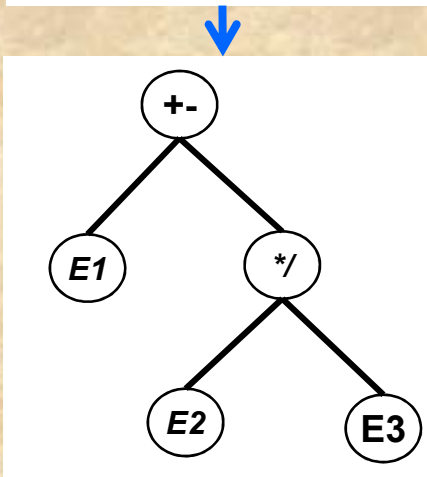
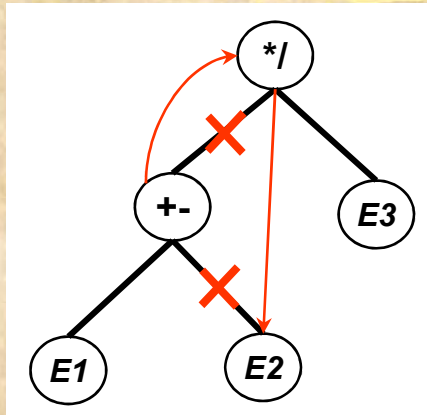
    if (!Parse_ServLAddr(&id, &port))
        return false;
    else
        if (!Parse_ServType(&type)) {
            SetParseError("'ServType' expected!");
            return false;
        }
    else
        (!Parse_ServAttrNameVals(attrs)) {

        ServiceDesc::ClearAttrs(attrs);
        delete[] type;

        SetParseError("'ServAttrNameVals' expected!");
        return false;
    }
```

Κώδικας από τη χρήση RDP για parsing σε πρωτόκολλα δικτύου (με ενσωματωμένα timeouts : αυτή είναι η μεγάλη ευελιξία ενός hand-made RDP).
Project 2WEAR, Savidis 2002-2003.

Συμπεράσματα (3/5)



Αναδιάταξη δέντρου εκφράσεων για επιβολή κανόνων προτεραιότητας (PT hacking)

```
Expression* SyntaxAnalyser::PrecedenceFiltering (Expression* expr) {
    if (expr->GetType() == Expression::ExprTypeMul) {
        MulExpression* mulExpr = (MulExpression*) expr;
        Expression* left = mulExpr->GetLeft();

        // Higher precedence ?
        //
        if (left->GetType() == Expression::ExprTypeAdd) {
            AddExpression* addExpr = (AddExpression*) left;
            Expression* right = addExpr->GetRight();

            mulExpr->SetLeft(right);
            addExpr->SetRight(mulExpr);
        }
        return addExpr;
    }
    else
        if (left->GetType() == Expression::ExprTypeSub) {
            SubExpression* subExpr = (SubExpression*) left;
            Expression* right = subExpr->GetRight();

            mulExpr->SetLeft(right);
            subExpr->SetRight(mulExpr);
        }
        return subExpr;
    }
    else
        return expr;
}
```

Μπορείτε εκ των υστέρων να επιβάλλετε κανόνες στα παραγόμενα δέντρα εάν προτιμάτε parse-tree hacking από grammar hacking

Συμπεράσματα (4/5)

- Το γεγονός ότι δεν έχουμε $LL(1)$ γραμματική δεν σημαίνει ότι δεν έχουμε δυνατότητα να κατασκευάσουμε έναν RDP
 - είτε με πρόβλεψη ή με backtracking
- Απλά σημαίνει ότι δεν μπορούμε να κατασκευάσουμε καθοδικό αναλυτή με πίνακα, καθώς θα έχει στοιχεία με πολλαπλές παραγωγές
 - Στην πράξη οι πολλαπλές παραγωγές σημαίνουν ότι ο αναλυτής θα έπρεπε να μπορεί να δοκιμάζει εναλλακτικούς γραμματικούς κανόνες και τελικά να αποδεχθεί ως σωστό όποιον πετύχει
 - Όμως ο βασικός αλγόριθμος που είδαμε για τον τρόπο λειτουργίας του καθοδικού αναλυτή με πίνακα δεν υποστηρίζει κάτι τέτοιο
- Ωστόσο μπορούμε να υποστηρίξουμε κάτι τέτοιο με κατάλληλο κάθε φορά *hacking* του RDP
 - Εν γένει ο RDP μπορεί να «πειραχθεί» όσο θέλετε για να μπορεί να αντιμετωπίζει προβλήματα τέτοιου είδους

Συμπεράσματα (5/5)

- Π.χ. για το γνωστό *if ...then ...else* πρόβλημα, γνωρίζοντας ότι πρέπει να επιβάλλουμε τον κανόνα
 - *το else να γίνει matched με το πλησιέστερο if*
 - αντί να παίζουμε με την γραμματική, δημιουργούμε μία parse συνάρτηση που να επιβάλλει αλγοριθμικά τον κανόνα

```
void expr_f (void);
void if_optelse_f (void);
void other_f (void);

void stmt_f (void) {
    if (lookAhead==IF_TOKEN) if_optelse_f();
    else other_f();
}
```

Μπορείτε εύκολα να αποδείξετε ότι δεν είναι δυνατόν κάποιο **else** να μην γίνει **matched** στην ίδια κλήση συνάρτησης (δηλ. στο υπό-δέντρο) με το πλησιέστερο του unmatched **if**

```
void if_optelse_f (void) {
    if (lookAhead == IF_TOKEN) {
        match(IF_TOKEN);
        match(OPENPAR_TOKEN);
        expr_f();
        match(CLOSEPAR_TOKEN);
        stmt_f();

        // Any trailing ELSE is always forced to match
        // with this IF algorithmically, not grammatically!
        // This is RDP hacking, not grammar hacking.
        //
        if (lookAhead == ELSE_TOKEN) {
            match(ELSE_TOKEN);
            stmt_f();
        }
    }
    else
        assert(0);
}
```